

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Organización de Lenguajes y Compiladores 2

Sección: A

Docente: Mario Moisés Ramírez Tobar



“Manual de Tecnico Proyecto1, Fase 1”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 26 de septiembre de

2026

1. Introducción

El presente documento es el Manual Técnico del Proyecto 1 “Contacto”, desarrollado para el curso Organización de Lenguajes y Compiladores 2. A diferencia de la Práctica 1 (Codex Latinus), que implementaba un único lenguaje, este proyecto compila TRES lenguajes fuente distintos, cada uno con su propio paradigma, y los une mediante un mecanismo de importación entre archivos.

El compilador está implementado en Java 21, utilizando ANTLR 4.13.2 para la generación de los analizadores léxico y sintáctico de los tres lenguajes, con una interfaz gráfica construida en Java Swing. A diferencia de la práctica anterior, este proyecto no se limita a analizar el código fuente: genera código C ejecutable real a partir de las cuartetas, verificado con gcc.

Los tres lenguajes son:

- Y? — lenguaje imperativo con estructuras y funciones, de sintaxis inspirada en Python (bloques por indentación, sin llaves).
- Zetariano — lenguaje orientado a objetos, de sintaxis inspirada en Java (clases, atributos, métodos, constructores).
- Pig Latin — lenguaje orquestador de sintaxis inspirada en latín, heredero directo del CodexLatinus de la Práctica 1. Importa clases de archivos .z y estructuras/funciones de archivos .y, y es el punto de entrada real de cualquier programa.

Esta arquitectura de tres lenguajes obliga a una decisión de diseño que marca todo el proyecto: los símbolos (clases, estructuras, funciones) de un archivo .z o .y deben ser visibles para el archivo .pig que los importa. Por eso el proyecto incluye un orquestador multi-archivo (sección 9) que no existía en la Práctica 1, donde todo el análisis ocurría sobre un único archivo.

Otra diferencia deliberada respecto a Codex Latinus: este proyecto NO construye un Árbol de Sintaxis Abstracta (AST) propio. El análisis semántico y la generación de código trabajan directamente sobre el árbol que produce ANTLR, usando el patrón Listener para el análisis semántico y el patrón Visitor para la generación de código (sección 6.1 explica la razón de este cambio de enfoque).

2. Arquitectura del Sistema

2.1 Estructura de paquetes

El proyecto sigue una arquitectura en capas bajo Maven. A diferencia de Codex Latinus, donde todos los paquetes pertenecían a un único lenguaje, aquí se distingue entre infraestructura COMPARTIDA (paquete comun) y el FRENTE de cada lenguaje (paquetes zetariano, ylang y piglatin), cada uno con sus propias subcarpetas semantico/ y generador/.

Infraestructura compartida (contacto.comun)

Paquete	Contenido
comun.tipos	TipoPrimitivo, Tipo, Operador, TablaTipos — sistema de tipos único para los 3 lenguajes
comun.simbolos	RolSimbolo, Simbolo, Ambito, TablaSimbolos — tabla de símbolos única para los 3 lenguajes
comun.errores	TipoError, ErrorCompilacion, RecolectorErrores, EscuchaErrores — manejo de errores

	centralizado
comun.cuartetas	Cuarteta (abstracta) y sus 9 subtipos, CodigoTransformable, GeneradorCuartetas
comun.cuartetas.acceso	Lugar (abstracta) y sus 4 subtipos: NombreLugar, LiteralLugar, CampoLugar, IndiceLugar
comun.codegen	ModeloPrograma, AmbitosGeneracion, RuntimeC, GeneradorCodigoC — traducción de cuartetas a C
comun.orquestador	OrquestadorPig — resuelve los import de un .pig y compila todo en conjunto
comun.ui	VentanaPrincipal, PanelEditor, ArbolArchivos, PanelSalida, ControladorCompilacion, ResaltadorSintaxis

Frente de cada lenguaje

Paquete	Contenido
contacto.zetariano	Gramática Zetariano.g4 (generada); zetariano.semantico.ZetarianoSemanticoListener; zetariano.generador.ZetarianoGeneradorCuartetas
contacto.ylang	Gramáticas YLangLexer.g4 / YLangParser.g4 (generadas); YLangLexerBase (indentación); ylang.semantico.YLangSemanticoListener; ylang.generador.YLangGeneradorCuartetas
contacto.piglatin	Gramática PigLatin.g4 (generada); piglatin.semantico.PigLatinSemanticoListener; piglatin.generador.PigLatinGeneradorCuartetas

2.2 Flujo de compilación

Compilar un programa en este proyecto significa siempre compilar un archivo .pig junto con todo lo que importa. El OrquestadorPig es quien coordina las etapas:

Etapas	Componente	Descripción
1	PigLatinLexer / Parser	Se analiza léxica y sintácticamente el .pig. Si hay errores, el proceso se detiene aquí.
2	OrquestadorPig	Se leen las rutas de import del .pig y se resuelven a archivos .z / .y reales en disco.
3	ZetarianoSemanticoListener / YLangSemanticoListener	Cada archivo importado se analiza léxica, sintáctica y semánticamente por separado, con el pipeline de SU propio lenguaje.
4	ModeloPrograma	Las clases, estructuras y funciones de TODOS los archivos importados se registran en un modelo único, compartido.
5	PigLatinSemanticoListener	El .pig se analiza semánticamente contra ese modelo: ya puede resolver novus Persona(...), pila.apilar(...), etc.
6	Generadores de cuartetas	Se generan las cuartetas de cada archivo importado y del .pig, todas contra el mismo ModeloPrograma.
7	GeneradorCodigoC	Se produce un único archivo .c con las estructuras, funciones y el main, listo para compilar con gcc.

Esta secuencia reemplaza, para un proyecto de varios archivos, el papel que en Codex Latinus cumplía la clase Compilador con un único archivo: aquí el orquestador es quien decide el orden y corta el proceso si una etapa falla, exactamente por la misma razón que en la práctica anterior — un análisis semántico construido sobre símbolos importados incompletos generaría errores falsos.

3. Los Tres Lenguajes Fuente

Antes de entrar al detalle técnico de cada fase, esta sección compara los tres lenguajes lado a lado. Los tres comparten los mismos cinco tipos primitivos y la misma jerarquía de conversión implícita (sección 6.2), pero cada uno los nombra distinto y cada uno resuelve el control de flujo con sus propias palabras reservadas.

Comparación general

	Y?	Zetariano	Pig Latin
Extensión	.y	.z	.pig
Paradigma	Imperativo estructurado	Orientado a objetos	Orquestador / script
Bloques	Indentación (INDENT/DEDENT)	Llaves { }	Llaves { } + finis
Define	Estructuras y funciones libres	Una clase por archivo	Nada propio: solo importa

Tipos primitivos equivalentes

Categoría	Y?	Zetariano	Pig Latin	Tipo interno
Entero	entero	int	numerus	TipoPrimitivo.ENTERO
Decimal	flotante	double	decimalis	TipoPrimitivo.DECIMAL
Carácter	caracter	char	littera	TipoPrimitivo.CARACTER
Cadena	cadena	String	textum	TipoPrimitivo.CADENA
Booleano	bool	boolean	bool	TipoPrimitivo.BOOLEANO

La columna “Tipo interno” es la clave de todo el sistema de tipos: cada lenguaje traduce su propio token de tipo a uno de los nueve valores del enum TipoPrimitivo (sección 6.2), y a partir de ahí las reglas de compatibilidad, la generación de cuartetos y la generación de C son exactamente las mismas para los tres lenguajes.

4. Análisis Léxico

4.1 Descripción general

Cada lenguaje tiene su propio analizador léxico, generado automáticamente por el plugin antlr4-maven-plugin a partir de su gramática. Los tres son sensibles a mayúsculas y minúsculas.

Una diferencia deliberada frente a Codex Latinus: en aquel proyecto los comentarios se descartaban con la instrucción skip, lo que significaba que el lexer nunca los entregaba en el flujo de tokens y, por lo tanto, el resaltador de sintaxis no podía pintarlos de un color distinto (tenía que buscarlos aparte, recorriendo el texto). En este proyecto los comentarios usan channel(HIDDEN) en vez de skip: el parser sigue sin verlos (el análisis sintáctico no cambia), pero el lexer sí los entrega a quien pida el flujo completo de tokens — que es exactamente lo que necesita el resaltador de sintaxis de la interfaz (sección 10.2) para pintarlos en cursiva sin tener que tocar el texto directamente.

4.2 Y?: indentación sensible al contexto

Y? es el único de los tres lenguajes cuyos bloques se delimitan por indentación en vez de llaves, igual que Python. ANTLR4 no soporta esto de forma nativa porque la indentación es sensible al contexto (no es un lenguaje libre de contexto en el sentido estricto), así que se resuelve con la misma técnica que usan las gramáticas de Python en el repositorio oficial grammars-v4:

- La gramática (YLangLexer.g4) reconoce un token crudo NEWLINE_RAW: el salto de línea más toda la indentación de la línea siguiente, pegados.
- La clase YLangLexerBase extiende Lexer y sobrescribe nextToken(): intercepta cada NEWLINE_RAW, compara el ancho de esa indentación contra una pila, y decide si emite

NEWLINE solo, NEWLINE + INDENT (subió el nivel) o NEWLINE + DEDENT... (bajó uno o más niveles).

- Una línea en blanco o de puro comentario no cambia la indentación: se detecta mirando el carácter siguiente en el flujo de entrada y se descarta sin emitir nada.
- Al llegar al fin de archivo se cierran todos los niveles de indentación abiertos con sus DEDENT correspondientes antes de emitir EOF.

4.3 Tokens principales por lenguaje

La lista completa de tokens de cada lenguaje está en su archivo .g4 (carpeta src/main/antlr4); esta sección resume las categorías más relevantes para entender el resto del documento.

Y? (YLangLexer.g4)

Categoría	Tokens
Secciones	SECCION_ESTRUCTURAS (%estructuras), SECCION_FUNCIONES (%funciones)
Declaración	ESTRUCTURA, DEFINIR
Control	SI, ENTONCES, SINO, CONTRARIO, ELEGIR, CASO, SIEMPRE, PARA, MIENTRAS, HACER, ROMPER, CONTINUAR, RETORNAR
E/S	IMPRIMIR, LEER
Literales	ENTERO_LITERAL, FLOTANTE_LITERAL, CADENA_LITERAL, CARACTER_LITERAL, VERDADERO, FALSO
Sintéticos	NEWLINE, INDENT, DEDENT (no existen como texto; los inyecta YLangLexerBase)

Zetariano (Zetariano.g4)

Categoría	Tokens
Declaración	PUBLIC, PRIVATE, CLASS, VOID, THIS
Control	IF, ELSE, SWITCH, CASE, DEFAULT, BREAK, FOR, WHILE, DO, CONTINUE, RETURN
Objetos	NEW, NULL
E/S	PRINTLN, PRINT, READLN
Literales	INT_LITERAL, DOUBLE_LITERAL, STRING_LITERAL, CHAR_LITERAL, TRUE, FALSE

Pig Latin (PigLatin.g4)

Categoría	Tokens
Secciones	VARIABLES, MAIOR, FIN_PROGRAMA (FINIS en mayúsculas)
Declaración	ESTO, SERIES, NOVUS, IMPORT
Control	SI, ALITER, DUM, FACERE, PER, PERGE, INTERRUPE, REDDERE, FINIS (minúsculas, cierra bloque)
Lógico	NON, AND, OR
Literales	ENTERO, DECIMAL, CADENA, CARACTER, VERUM, FALSUS

En los tres lenguajes las palabras reservadas se declaran antes que la regla ID, por la misma razón documentada en Codex Latinus: ANTLR resuelve los empates de igual longitud a favor de la regla declarada primero, así que si el orden fuera el inverso toda palabra reservada se reconocería como un identificador cualquiera.

4.4 Manejo de errores léxicos

La clase EscuchaErrores (comun.errores) es compartida por los tres lenguajes y por el orquestador. A diferencia de Codex Latinus —donde solo existía un archivo por compilación—, aquí un mismo

RecolectorErrores puede recibir errores de varios archivos en una sola pasada (el .pig y cada .z / .y que importa), así que cada error guarda también el nombre del archivo de origen, no solo la línea y la columna.

5. Gramáticas de los Tres Lenguajes

5.1 Estructura general de cada programa

Las tres gramáticas están escritas con la misma convención de notación que Codex Latinus: no terminales en minúscula camel, terminales en mayúscula, y alternativas etiquetadas con # para que ANTLR genere un método distinto en el Listener o el Visitor.

Y? (YLangParser.g4)

```
programa
    : NEWLINE* seccionEstructuras? seccionFunciones EOF
    ;

seccionEstructuras
    : SECCION_ESTRUCTURAS NEWLINE estructura+
    ;

seccionFunciones
    : SECCION_FUNCIONES NEWLINE funcionDef+
    ;
```

Zetariano (Zetariano.g4)

```
programa
    : clase EOF
    ;

clase
    : PUBLIC CLASS ID LBRACE miembro* RBRACE
    ;
```

Pig Latin (PigLatin.g4)

```
programa
    : seccionImports? seccionVariables? seccionMaior EOF
    ;

seccionImports
    : (IMPORT rutaImport)*
    ;

seccionMaior
    : MAIOR MAYOR instruccion* FIN_PROGRAMA? PYC?
    ;
```

Un detalle verificado contra un archivo de prueba real compartido por la auxiliar del curso: el cierre FIN_PROGRAMA de la sección MAIOR es opcional en la práctica, aunque el enunciado original lo muestra como obligatorio. La gramática lo acepta de forma opcional para admitir ambos casos sin rechazar programas válidos.

5.2 Filosofía de diseño: gramática permisiva, semántico preciso

Las tres gramáticas heredan directamente un principio de diseño de Codex Latinus: cuando una construcción es sintácticamente ambigua entre “válida” e “inválida”, la gramática la acepta y es el

analizador semántico quien decide, porque un mensaje semántico puede ser específico (“la variable x ya fue declarada en este ámbito”) mientras que un error sintáctico genérico solo dice “se esperaba tal token”.

El ejemplo más claro de esto en el proyecto actual es el lado izquierdo de una asignación: en los tres lenguajes, la gramática acepta cualquier expresión como destino de una asignación (objetivo IGUAL expresion en Pig Latin, expresion (operadorAsignacion expresion)? en Zetariano y Y?), aunque solo un identificador, un acceso a campo o un acceso a arreglo son destinos válidos de verdad. Es el analizador semántico (método esDestinoValido en cada Listener) quien rechaza, por ejemplo, asignar a un literal.

5.3 Precedencia de operadores

En los tres lenguajes, ANTLR4 resuelve la recursividad por la izquierda y asigna la precedencia según el orden de las alternativas dentro de la regla expresion: la primera alternativa tiene la mayor precedencia. La tabla siguiente resume el orden, de mayor a menor, común a los tres lenguajes (Pig Latin y Zetariano agregan el operador ternario, ausente en Y?):

Nivel	Operadores	Notas
1	()	Agrupación explícita
2	- unario, ! / non	Unarios
3	++, --	Incremento / decremento (prefijo y sufijo en Y? y Zetariano; solo sufijo en Pig Latin)
4	*, /, %	Multiplicativos (% solo en Zetariano)
5	+, -	Aditivos. El + también concatena cadenas
6	<, >, <=, >=	Relacionales, producen booleano
7	==, !=	Igualdad, producen booleano
8	&&	Conjunción lógica
9		Disyunción lógica
10	? :	Ternario (solo Zetariano)

5.4 Diferencias notables entre los tres

Pig Latin tiene una particularidad de forma que lo distingue de los otros dos: la regla objetivo (id.campo[indice].metodo()) es una lista PLANA de sufijos (objetivo: ID sufijoAcceso*), no una regla recursiva como expresion en Y? y Zetariano. Esto significa que su tipo no se puede resolver con el patrón habitual de abajo hacia arriba sobre el árbol: hay que iterar la lista de sufijos manualmente, acumulando el tipo paso a paso (ver ZetarianoSemanticoListener.exitObjetivo, sección 6.4).

Zetariano admite llamar a un método de la propia clase sin escribir this. explícito (metodo()) además de this.metodo()), igual que en Java. Esto se resolvió agregando una alternativa ID LPAREN argumentos? RPAREN a la regla expresion, tratada en el analizador semántico exactamente como si fuera this.metodo().

Y? admite literales compuestos posicionales para instanciar una estructura (Punto p1 = {10, 20, 85.5}): el analizador semántico valida esos valores contra los campos de la estructura EN EL ORDEN en que fueron declarados, no por nombre.

6. Análisis Semántico

6.1 Por qué no hay un AST propio

Codex Latinus construía un Árbol de Sintaxis Abstracta propio (paquete `ast`, con `Nodo` como clase base y más de 30 tipos de nodo entre expresiones e instrucciones) mediante un Listener (`ConstructorAST`) que transformaba el árbol de ANTLR en esos objetos. Ese diseño tiene una ventaja real: el AST es mucho más compacto que el árbol de análisis, porque descarta paréntesis, comas y punto y coma que no aportan significado.

Este proyecto toma una decisión distinta y la mantiene de forma consistente en los tres lenguajes: el análisis semántico y la generación de código trabajan directamente sobre el árbol que ANTLR ya genera. La razón es doble. Primero, ANTLR reparte automáticamente cada construcción gramatical a su propio método del Listener (`exitDeclaracionVariable`, `exitExpAditiva`, etc.) —eso ES el despacho polimórfico que un AST propio simularía a mano con cadenas de `instanceof`, y evitar esas cadenas era un objetivo explícito de este proyecto tras una observación sobre código “sucio” en una entrega anterior. Segundo, con tres lenguajes en vez de uno, mantener tres jerarquías de AST completas (más sus tres Listener constructores) hubiera triplicado el código sin agregar información nueva.

La contrapartida, documentada honestamente: las clases `XxxContext` que genera ANTLR traen mezcladas cosas propias del parser (índices de regla, posición de tokens) en vez de ser objetos limpios de dominio, como sí lo eran los nodos de Codex Latinus. Para un proyecto que necesitara reescribir o transformar el árbol (no solo leerlo), un AST propio volvería a ser la mejor opción.

6.2 Sistema de tipos compartido

Los tres lenguajes comparten una única jerarquía de tipos, definida en el paquete `comun.tipos`. La clase `TipoPrimitivo` enumera nueve categorías: los cinco tipos primitivos de la sección 3, más `ESTRUCTURA` (clase u objeto), `NULO` (el literal `null` / `NULL`), `VACIO` (sin retorno) y `ERROR` (comodín para no arrastrar errores en cascada).

La clase `Tipo` representa el tipo completo de una expresión, variable, campo o retorno: el primitivo, el nombre de la estructura cuando aplica, y la `PROFUNDIDAD` de arreglo (0 si no es arreglo, 1 para un arreglo simple, 2 para un arreglo de arreglos, etc., en vez de una sola bandera booleana). Este diseño se adoptó específicamente para soportar arreglos multidimensionales con inicialización anidada, confirmados por la auxiliar del curso en el foro de dudas.

La jerarquía de conversión implícita, verificada contra la clase `TablaTipos`, es la siguiente: un entero se ensancha a decimal automáticamente; la operación inversa NO ocurre, porque perdería información de forma silenciosa. Una cadena concatenada con cualquier otro tipo mediante `+` produce siempre cadena. `null` es asignable a cualquier tipo `ESTRUCTURA`.

6.2.1 Tabla de compatibilidad de tipos

Operación	Operandos	Resultado
+	cadena con cualquier tipo	cadena, por concatenación
+, -, *, /, %	numéricos entre sí	el de mayor nivel de los dos (decimal > entero)
-, *, /, %	cadena con cualquier tipo	error
<, >, <=, >=	numéricos entre sí	booleano
<, >, <=, >=	cadena o booleano involucrados	error
==, !=	numéricos entre sí, o tipos iguales	booleano
==, !=	null contra una estructura	booleano
&&,	booleano con booleano	booleano
&&,	cualquier otra combinación	error
! / non	booleano	booleano
- unario	numérico	el mismo tipo del operando
++, --	numérico	el mismo tipo del operando
cualquiera	arreglo o estructura completa	error (hay que acceder a una posición o un campo)

6.3 Tabla de símbolos compartida

Al igual que el sistema de tipos, la tabla de símbolos (paquete comun.simbolos) es una sola implementación reutilizada por los tres lenguajes. La clase Simbolo almacena nombre, tipo, rol (enum RolSimbolo: VARIABLE, PARAMETRO, FUNCION, METODO, CONSTRUCTOR, CLASE, ESTRUCTURA, ARREGLO) y una referencia genérica al nodo del árbol de ANTLR donde se definió, para que el generador de cuartetas pueda volver a él sin buscarlo de nuevo.

La clase Ambito representa un nivel de alcance, encadenado a su padre; TablaSimbolos gestiona la pila de ámbitos con entrarAmbito() / salirAmbito() / declarar() / buscar(), y además mantiene un historial plano de TODO lo declarado durante el análisis, incluso de ámbitos ya cerrados. Ese historial es el que consume GeneradorCodigoC (sección 8.1) para declarar en C cada variable con su tipo real.

6.4 Particularidades semánticas por lenguaje

Zetariano registra los miembros de una clase (campos, constructor, métodos) en un pre-registro dentro de enterClase, ANTES de revisar ningún cuerpo. Sin este pre-registro, un método no podría llamar a otro declarado más abajo en el mismo archivo —exactamente el mismo problema, y la misma solución, que Codex Latinus documenta para las firmas de función en su pasada 2 (sección 6.3 del manual de la Práctica 1).

Y? aplica el mismo principio a sus dos secciones: enterSeccionFunciones pre-registra la firma de cada función antes de revisar cuerpos, y enterSeccionEstructuras pre-registra el nombre de cada estructura antes de revisar sus campos. A diferencia de Zetariano —donde todos los campos de la única clase del archivo viven en un solo ámbito plano—, en Y? puede haber varias estructuras, cada una con sus propios campos (que incluso pueden repetirse entre estructuras distintas). Por eso un acceso obj.campo no busca en la tabla de símbolos plana: busca la definición de la estructura de obj (guardada como nodo de ANTLR en el propio Simbolo) y recorre sus campos ahí.

Pig Latin es el caso más distinto de los tres: no define sus propias estructuras, clases ni funciones, así que su analizador semántico recibe un `ModeloPrograma` ya construido con todo lo importado (sección 8.1) en vez de construir su propia tabla desde cero. Su regla objetivo tampoco es recursiva (sección 5.4), así que su tipo se resuelve iterando la lista de sufijos manualmente, en vez de con el patrón bottom-up habitual sobre árboles recursivos.

6.5 Validaciones implementadas

Categoría	Verificaciones
Declaraciones	Símbolo no declarado. Nombre repetido en el mismo ámbito. Parámetros repetidos.
Tipos	Compatibilidad según la tabla de la sección 6.2.1. Conversión implícita válida. Retorno del tipo declarado.
Estructuras / clases	Campos sin repetir. Atributos inexistentes. Constructor con la aridad correcta (en la propia clase; ver limitación en 9.3 para clases importadas).
Arreglos	Índice numérico. Dimensión numérica en <code>new Tipo[n]</code> . Elementos de un literal {...} del mismo tipo.
Funciones / métodos	Cantidad de argumentos contra la firma. Tipo de cada argumento.
Control de flujo	Condiciones estrictamente booleanas en <code>si/if</code> , <code>mientras/while</code> , <code>dum</code> , <code>facere</code> , <code>per</code> , <code>para</code> . <code>break/switch</code> y <code>continue/ciclo</code> válidos solo en su contexto. <code>return/reddere</code> solo dentro de una función.
Entrada y salida	El destino de una lectura debe ser asignable. No se puede leer ni imprimir un arreglo o una estructura completa.

7. Generación de Código de Tres Direcciones (Cuartetas)

7.1 Por qué Visitor y no Listener para el generador

El análisis semántico de los tres lenguajes se implementó como `Listener` (sección 6), pero el generador de cuartetas de los tres se implementó como `Visitor`. La razón es una limitación real del patrón `Listener`: recorre automáticamente todos los hijos de un nodo, en orden, antes de que el propio nodo padre pueda reaccionar. Eso es exactamente lo que hace falta para calcular tipos de abajo hacia arriba (el orden no importa para la corrección), pero es exactamente lo que estorba para generar saltos de un condicional o un ciclo, donde el orden importa: primero se evalúa la condición, luego se emite el salto, y SOLO DESPUÉS se visita el cuerpo, y al final se coloca la etiqueta de salida. Un `Visitor` deja que el propio código decida cuándo llamar `visit(...)` sobre cada hijo, que es justo el control que se necesita.

7.2 Representación de un operando: Lugar

Cada cuarteta necesita representar DÓNDE vive un valor: una variable, un temporal, un literal, un campo de un objeto o un elemento de un arreglo. La primera versión de este proyecto usaba texto plano (`String`) para eso; se reemplazó por una jerarquía de clases (`paquete comun.cuartetas.acceso`) inspirada en el patrón `MemoryAccess` de un compañero de curso, adaptado y simplificado para este proyecto:

Clase	Representa
Lugar (abstracta)	Contrato común: <code>generarC(StringBuilder)</code> e implementa <code>CodigoTransformable</code>
NombreLugar	Una variable o un temporal (<code>t0</code> , <code>t1</code> , ...) —en C ambos son, simplemente, un identificador
LiteralLugar	Un literal ya en su forma de texto C (<code>10</code> , <code>"hola"</code> , <code>3.5</code>)
CampoLugar	<code>base.campo</code> , generado como <code>base->campo</code> en C (los objetos viven en heap)

IndiceLugar	base[indice], igual en C que en el lenguaje fuente
-------------	--

Cada Cuarteta y cada Lugar implementan la interfaz `CodigoTransformable` (un solo método: `generarC(StringBuilder)`). Esto significa que cada clase sabe traducirse a sí misma a C, en vez de que exista un generador externo con un switch gigante sobre “qué tipo de cosa es esto” —el mismo principio de diseño que evitó las cadenas de `instanceof` en el análisis semántico (sección 6.1).

7.3 Tipos de cuarteta

Clase	Forma en C generado
AsignacionCuarteta	destino = origen;
OperacionCuarteta	destino = izq OP der; (o destino = OP izq; si es unaria)
EtiquetaCuarteta	nombre;;
SaltoCuarteta	goto etiqueta;
SaltoCondicionalCuarteta	if (condicion) / if (!(condicion)) goto etiqueta;
ImprimirCuarteta	printf("formato", valor);
LeerCuarteta	scanf("formato", &destino);
LlamadaCuarteta	[destino =] nombre(objetivo?, args...);
RetornoCuarteta	return [valor];

7.4 Patrones de generación de control de flujo

Los tres generadores (Zetariano, Y?, Pig Latin) usan el mismo esquema de etiquetas para cada construcción de control de flujo, con nombres descriptivos generados por `GeneradorCuartetas.nuevaEtiqueta()`:

```
mientras / while / dum:
    L_inicio:
        if_false condicion goto L_fin
        <cuerpo>
        goto L_inicio
    L_fin:

hacer-mientras / do-while / facere:
    L_inicio:
        <cuerpo>
    L_continua:
        if_true condicion goto L_inicio
    L_fin:

para / for / per:
    <inicializacion>
    L_inicio:
        if_false condicion goto L_fin
        <cuerpo>
    L_continua:
        <actualizacion>
        goto L_inicio
    L_fin:
```

La distinción entre `L_inicio` y `L_continua` es la que permite que `continue` / `perge` (que deben re-evaluar la actualización antes de la condición) y `break` / `interrumpe` (que deben saltar directo al final) apunten a lugares distintos. Cada generador mantiene una pila de pares {etiquetaContinua, etiquetaFin} con el ciclo más interno activo para resolver estas dos instrucciones sin importar cuántos ciclos estén anidados.

8. Generación de Código C

Esta es la capacidad que distingue a este proyecto de la Práctica 1: las cuartetas no se quedan como una representación intermedia para depuración, sino que se traducen a un archivo `.c` completo, compilable con `gcc`. El paquete `comun.codegen` concentra esta traducción en cuatro clases.

8.1 ModeloPrograma

Es el “linker” del proyecto: une la información de TODOS los archivos que participan en una compilación (el `.pig` y cada `.z` / `.y` que importa) antes de generar una sola línea de C. Contiene tres clases anidadas:

- Variable — nombre fuente, nombre ya adaptado a C, y tipo.
- Estructura — nombre, si es clase (Zetariano) o estructura (Y?), y su lista de campos.
- Funcion — nombre fuente y en C, la clase a la que pertenece (null si es libre), si es constructor o el main, tipo de retorno, parámetros, y el RANGO de cuartetas que forman su cuerpo (para que `GeneradorCodigoC` sepa exactamente qué cuartetas van dentro de cada función C).

El método `asignarNombresC()` resuelve un problema real de generar C desde varios lenguajes con distinta terminología: dos clases importadas podrían tener un método con el mismo nombre. La solución es un esquema de “name mangling” consistente (`nombreSeguroC`), y para la sobrecarga (mismo nombre, distinta aridad) se agrega un sufijo numérico (por ejemplo, `Contador_sumar_1` y `Contador_sumar_2`).

8.2 AmbitosGeneracion

Resuelve un problema de nombres a nivel de UNA función: si dos bloques distintos dentro del mismo método declaran una variable con el mismo nombre fuente pero tipos distintos, C no lo permite en el mismo ámbito de función (todas las locales se declaran juntas al inicio, ver 8.4). `AmbitosGeneracion` lleva una pila de ámbitos léxicos igual que `TablaSimbolos` (sección 6.3), pero en vez de resolver tipos resuelve el nombre C real de cada variable, renombrando con un sufijo (`x`, `x_2`) cuando hace falta.

8.3 RuntimeC

Un programa en Y?, Zetariano o Pig Latin necesita operaciones que C no ofrece de forma nativa: reservar memoria para un objeto o un arreglo dinámico, concatenar cadenas, convertir un número a texto para imprimirlo junto a una cadena, o leer una línea completa de la entrada estándar de forma segura. `RuntimeC` contiene ese código C como una constante de texto (CODIGO) que se copia al inicio de todo archivo generado, con funciones como `zc_reservar`, `zc_concatenar`, `zc_nuevo_arreglo`, `zc_leer_entero` y `zc_verificar_objeto`. Todas se declaran `static inline` para que `gcc -Wall` no avise por las que un programa en particular no llegue a usar.

La verificación `zc_verificar_objeto` es la que produce, en tiempo de ejecución, el mensaje “Error en tiempo de ejecución: se llamó a Clase.metodo sobre un objeto nulo” en vez de un `segmentation fault` cuando el programa intenta usar un objeto que nunca se inicializó —el caso más común es un `desapilar()` sobre una pila vacía cuando el propio programa fuente no revisó esa condición antes.

8.4 GeneradorCodigoC

Con el ModeloPrograma completo, GeneradorCodigoC.generar() arma el archivo final en este orden: el runtime (RuntimeC.CODIGO), un typedef struct por cada clase o estructura, los prototipos de todas las funciones, una función de C por cada constructor, método y función libre (recorriendo su rango de cuartetas y llamando cuarteta.generarC() en cada una), y por último el main del .pig.

Cada función declara al inicio TODAS sus variables locales y temporales, con su tipo real, inicializadas en 0 —una decisión necesaria porque en C un goto no puede saltar por encima de la declaración de una variable con inicializador; declarando todo de una vez, antes de cualquier goto generado por un if o un ciclo, ese problema desaparece.

8.5 Convenciones del código generado

Convención	Ejemplo
Objetos y estructuras viven en heap	Nodo* new_Nodo(int dato1) —siempre puntero
Constructor	Nodo* new_Nodo(int dato1) (uno implícito si la clase no define ninguno)
Método	int Nodo_getDato(Nodo* this)
Función libre (Y?)	void imprimirBienvenida(void)
Campo sin this. dentro de un método	se traduce a this->campo
Sobrecarga	se agrega N: Contador sumar_1, Contador sumar_2
Objeto nulo	se verifica al entrar a cada método; termina el programa con código 1 y un mensaje, no con segmentation fault

9. Orquestador Multi-Archivo

9.1 Resolución de imports

La sección import de un archivo .pig usa una notación con puntos para carpetas (utils.utils.y equivale a la ruta utils/utils.y relativa a la carpeta del .pig). OrquestadorPig.resolverRuta() separa la extensión final (.z o .y) y convierte el resto de puntos en separadores de carpeta.

9.2 Flujo de compilación conjunta

Ya descrito en la sección 2.2: cada archivo importado se compila con el pipeline completo de SU lenguaje (léxico, sintáctico, semántico), sus símbolos se agregan al ModeloPrograma compartido, y solo entonces se analiza semánticamente el .pig contra ese modelo ya completo.

9.3 Limitaciones conocidas

Documentadas aquí de forma explícita, no como un descuido: el “name mangling” de una llamada a método (sección 8.1) no siempre puede distinguir con certeza absoluta a qué clase pertenece un método cuando dos clases importadas comparten nombre y aridad de método; y la validación de constructores para una clase distinta a la que se está analizando en un momento dado depende de que esa clase ya haya sido registrada en el ModeloPrograma antes de necesitarla, lo cual el orden real de import garantiza en los casos probados (sección 13) pero no está demostrado para cualquier orden posible de imports circulares.

10. Interfaz Gráfica

10.1 Componentes principales

Componente	Función
VentanaPrincipal	Ventana principal (JFrame): árbol de archivos, pestañas de editor, consola de salida, menú y botón Compilar
PanelEditor	Una pestaña del editor: JTextPane con StyledDocument, resaltado de sintaxis en vivo
ArbolArchivos	Árbol de trabajo (JTree) sobre el sistema de archivos real: abrir, nuevo archivo/carpeta, eliminar, renombrar, refrescar
PanelSalida	Consola con tres pestañas: Errores, Cuartetas y Código C
ControladorCompilacion	Conecta el botón Compilar con OrquestadorPig (para .pig) o con una verificación léxica/sintáctica (para un .z/.y suelto)
ResaltadorSintaxis	Interfaz común; tres implementaciones (Zetariano, Y?, Pig Latin), una por lenguaje

10.2 Coloreado de sintaxis

Igual que en Codex Latinus, el coloreado NO emplea expresiones regulares ni ninguna librería externa: cada ResaltadorXxx reutiliza el lexer de ANTLR de su propio lenguaje. Cada token que produce el lexer trae su tipo y sus posiciones inicial y final, que es precisamente la información necesaria para pintar el texto; si en el futuro se agrega una palabra reservada a una gramática, el editor la reconoce sin modificar una sola línea del resaltador.

A diferencia de Codex Latinus —donde los comentarios se descartaban con skip y había que detectarlos buscando directamente en el texto—, aquí los comentarios usan channel(HIDDEN) (sección 4.1), así que el propio lexer los entrega como cualquier otro token y el resaltador los pinta en cursiva sin ningún caso especial.

Un defecto real detectado durante las pruebas (sección 13) obligó a un ajuste importante: leer el texto a colorear desde el Document de Swing y no desde JTextPane.getText(). Los archivos de prueba, creados en Windows, usan fin de línea CRLF; JTextPane.getText() devuelve esos \r\n tal cual, mientras que el Document interno de Swing normaliza a un solo \n por línea. Tokenizar con el texto crudo desalineaba las posiciones un carácter más por cada línea, así que el color se veía “corrido” y el desfase crecía a medida que avanzaba el archivo.

Paleta de colores

Categoría	Estilo
Palabras reservadas	Azul, negrita
Tipos de datos	Verde agua
Cadenas y caracteres	Naranja
Números	Verde claro
Comentarios	Verde oscuro, cursiva
Identificadores y el resto	Color de texto normal (los operadores y signos de puntuación no se colorean aparte)

10.3 Árbol de trabajo

ArbolArchivos reconstruye el árbol completo cada vez que algo cambia (sencillo y suficiente para el tamaño de proyecto de este curso, sin carga perezosa). Permite abrir un archivo con doble clic, y con el menú

contextual: nuevo archivo, nueva carpeta, renombrar, eliminar y refrescar. Después de compilar, la ventana refresca el árbol automáticamente para que el .c recién generado aparezca sin acción manual.

10.4 Compilación desde la interfaz

El botón Compilar, antes de invocar a `ControladorCompilacion`, guarda TODAS las pestañas modificadas (no solo la activa): si se está editando `Pila.z` sin guardar y se compila `main.pig`, la compilación debe reflejar lo que se ve en el editor de `Pila.z`, no una versión vieja que haya quedado en disco. Al terminar, el código C se muestra en la pestaña correspondiente y además se guarda en disco junto al .pig (mismo nombre, extensión .c).

11. Herramientas y Configuración del Proyecto

11.1 Tecnologías utilizadas

Tecnología	Versión	Uso en el proyecto
Java	21	Lenguaje de implementación del compilador y la interfaz
ANTLR	4.13.2	Generación de los tres analizadores léxicos y sintácticos
antlr4-maven-plugin	4.13.2	Integra la generación en el ciclo de construcción de Maven
Apache Maven	3.x	Gestión de dependencias y construcción
Java Swing	incluido en el JDK	Interfaz gráfica de escritorio (editor, árbol, consola)
gcc	compatible con C11	Compilación del código C generado (verificado con -Wall -Wextra)
Git / GitHub	—	Control de versiones y entrega

11.2 Configuración de Maven

El `pom.xml` declara la dependencia del runtime de ANTLR y configura el plugin para que genere tanto Listener como Visitor —ambos hacen falta: el primero para los tres analizadores semánticos, el segundo para los tres generadores de cuartetos (sección 7.1):

```
<properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <antlr4.version>4.13.2</antlr4.version>
</properties>

<dependency>
  <groupId>org.antlr</groupId>
  <artifactId>antlr4-runtime</artifactId>
  <version>${antlr4.version}</version>
</dependency>

<plugin>
  <groupId>org.antlr</groupId>
  <artifactId>antlr4-maven-plugin</artifactId>
  <version>${antlr4.version}</version>
  <configuration>
    <listener>true</listener>
    <visitor>true</visitor>
  </configuration>
  <executions>
    <execution>
```

```

        <goals><goal>antlr4</goal></goals>
      </execution>
    </executions>
  </plugin>

```

La ubicación de cada gramática determina el paquete de sus clases generadas: al estar en `src/main/antlr4/contacto/zetariano`, `src/main/antlr4/contacto/ylang` y `src/main/antlr4/contacto/piglatin`, las clases resultantes pertenecen a los paquetes `contacto.zetariano`, `contacto.ylang` y `contacto.piglatin` respectivamente.

11.3 Construcción y ejecución

Para compilar el proyecto completo:

```
mvn clean compile
```

El uso de `clean` es obligatorio cada vez que se modifica cualquiera de las tres gramáticas: el plugin realiza una construcción incremental y omite el reprocesamiento si no detecta cambios en el archivo `.g4`, lo que puede llevar a compilar contra un analizador desactualizado.

Para ejecutar la interfaz gráfica:

```
java -cp target/classes contacto.comun.ui.VentanaPrincipal [carpeta-inicial]
```

Para compilar un programa desde la línea de comandos, sin la interfaz:

```
java -cp target/classes contacto.comun.orquestador.PruebaOrquestador
ruta/al/archivo.pig
```

11.4 Estructura de carpetas del repositorio

```

Proyecto_1_Fase_1_Compiladores_2_Mynor_Monzon_2S_2026/
  pom.xml
  src/main/antlr4/contacto/{zetariano,ylang,piglatin}/ ->Gramaticas
  src/main/java/contacto/
    comun/{tipos,simbolos,errores,cuartetos,codegen,orquestador,ui}/
    zetariano/{semantico,generador}/
    ylang/{semantico,generador}/
    piglatin/{semantico,generador}/
  test-programs/pila/ -> main.pig, Nodo.z, Pila.z, utils/utils.y

```


12. Diagrama de Clases

El diagrama de clases completo se entrega como archivo aparte (diagrama-clases.drawio), editable en app.diagrams.net o en la aplicación de escritorio de draw.io. Agrupa las 42 clases principales del proyecto por paquete —los mismos paquetes descritos en la sección 2.1— y muestra las relaciones de herencia, implementación de interfaces, composición y uso entre ellas.

[Espacio reservado para insertar aquí la imagen exportada del diagrama: abrir diagrama-clases.drawio y usar File > Export as > PNG.]

13. Pruebas Realizadas

A diferencia de Codex Latinus, este proyecto no cuenta todavía con una batería curada de archivos de prueba organizada en carpetas válidos/errores como la de la Práctica 1. La verificación realizada fue más específica, orientada a confirmar que el flujo completo —los tres lenguajes, el orquestador y la generación de C— funciona de punta a punta; se deja como pendiente honesto ampliarla antes de una entrega final.

13.1 Programa principal de prueba: la Pila

Un programa compartido por la auxiliar del curso como referencia, compuesto por cuatro archivos: `main.pig` (orquestador con un menú interactivo), `Nodo.z` y `Pila.z` (clases de Zetariano, una pila enlazada) y `utils/utils.y` (funciones de Y?).

- Compilación: OrquestadorPig procesa los cuatro archivos sin errores.
- Código C: se compiló con `gcc -std=c11 -Wall -Wextra` sin ningún error ni advertencia.
- Ejecución real: `apilar 5`, `apilar 7`, `imprimir` e `imprime` “Pila [Cima -> Base]: 7, 5”; `desapilar` imprime “Elemento desapilado: 7”; `salir` imprime “Fin del programa”.
- Casos límite: una entrada no numérica se toma como 0 en vez de dejar el programa en un ciclo infinito de lectura; `desapilar` con la pila vacía termina con “Error en tiempo de ejecución: se llamó a `Nodo.getDato` sobre un objeto nulo” en vez de un `segmentation fault` (el propio `Pila.z` no revisa si está vacía antes de `desapilar`; en Java también fallaría).

13.2 Programa de prueba ampliado

Un segundo programa, con un archivo de cada lenguaje, se escribió específicamente para ejercitar construcciones no cubiertas por el ejemplo de la pila: `ciclo for/per`, `switch/elegir` con cadenas, `do-while/facere`, arreglos, sobrecarga de métodos, una estructura de Y? con un arreglo como campo, y literales {...}. Todos los resultados de esta prueba coincidieron con los valores calculados a mano.

13.3 Verificación de la interfaz gráfica

- La ventana abre correctamente y el árbol de archivos muestra la estructura real de `test-programs/pila`.
- El coloreado se verificó visualmente como correcto en los tres lenguajes (palabras clave, tipos, cadenas, números y comentarios cada uno en su color).
- El botón `Compilar` sobre `main.pig` muestra “Compilación OK”, las cuartetas de los cuatro archivos, el código C completo, y guarda ese C en disco.

- Cerrar una pestaña con cambios sin guardar muestra un diálogo de confirmación; guardar todas las pestañas modificadas antes de compilar se verificó editando Pila.z sin guardar y confirmando que el C generado reflejaba ese cambio.

13.4 Defectos detectados y corregidos durante el desarrollo

Se documentan aquí, en el mismo espíritu que la sección equivalente del manual de la Práctica 1, porque el proceso de encontrarlos fue parte real de la verificación del sistema:

- El análisis semántico de Zetariano registraba cada miembro de la clase en el orden en que aparecía en el archivo, así que un método no podía llamar a otro declarado más abajo —válido en Java. Se corrigió pre-registrando todos los miembros en enterClase.
- switch (Zetariano) y elegir (Y?) ejecutaban el caso por defecto incluso cuando otro caso ya había coincidido antes, en vez de seguir el fall-through real de Java/C.
- El análisis semántico de Pig Latin mezclaba las variables locales de los métodos importados con las variables del propio .pig en una sola tabla plana, y no encontraba los campos de las estructuras de Y?. Se corrigió resolviendo cada símbolo contra su clase o estructura exacta (ModeloPrograma, sección 8.1) en vez de una búsqueda plana.
- YLangLexerBase no emitía el NEWLINE que cierra la última línea de un archivo antes de los DEDENT finales, así que cualquier archivo .y cuya última instrucción no terminara en salto de línea fallaba el análisis sintáctico completo.
- En la gramática de Pig Latin, el >> entre valores de una instrucción de impresión era opcional; combinado con el ; también opcional, una línea de lectura siguiente se interpretaba como un valor más de la impresión anterior, y el menú del programa de prueba quedaba en un ciclo infinito. Se corrigió haciendo obligatorio el >> entre valores.
- El desfase de coloreado por CRLF descrito en la sección 10.2.